Building Web and Mobile Apps WS2025/26 (Prof. Dr. Cathrin Möller)

Exercise 3 - 10.11./11.11.2025

Practical Exercises

Do not hesitate to ask questions or if you want feedback for your coding style.

Please commit and push to GitLab regularly and ensure @*cathrin.moeller* and my two tutors @pinky-jitendra.tinani and @rahul.patil are invited as a reporter.

Learning goals for today

Today we want to learn **connecting the ExpressJS backend to our frontend** using the fetch API to load JSON data.
We will practice creating matching APIs and matching **typed data interfaces**.
We also want to practice using the **ES6 .map() and .filter()** methods.
We want to create a **typeahead** search from an input to the backend.

1) Reuse the **ExpressJS app** you created last week and ensure you have an endpoint that **returns a list of users and corresponding addresses from a local JSON file**. The user/address data have separate attributes for firstName, lastName, phoneNumber, eMail, street, houseNumber, postCode and city.

Run the backend via „node app.js“ or „node index.js“ and make sure it is running at port 8081.
You should be able to see the list of users with addresses accessing **localhost:8081/users** in your browser.

Now create the matching frontend code that will fetch and display the data. Create one HTML file (users.html, in vanilla or a new folder) referencing a JS script src. The referenced JS file should be originally a TS file that can be translated via „tsc users.ts --lib dom,es2015“. You can add a script in package.json for that.
Now inside your users.ts add a **fetch API** call to the typescript code **loading the users from the backend**. The argument for fetch() is the URL of the Backend.

Add some console.log or other debugging to the .then() callback after the fetch API call to see if the JSON with the user data is received.
Run the frontend from a different port (than the backend) on localhost (e.g. 8080 or 4200). You can inspect the request from FE to BE in your browser (e.g. network tab). You will observe CORS errors. Now you have multiple options fixing the problem in order to allow the FE to receive data from your backend.

Option 1) You could do it manually, setting the HTTP Response Header and returning status 200 in case of an OPTIONS request.
Hint: in expressJS you can access the incoming request method via „**req.method**“. You can set HTTP Headers via **res.header(‚NAME‘, ‚VALUE‘)**. And you can reply with more than one response using if/else..

Option 2) In ExpressJS, there is middleware available.
You can check https://expressjs.com/en/resources/middleware/cors.html for usage.

2) Now use the received User data and **map** it to the **typed interfaces** you created in an earlier exercise.
Create an Array of Users in TS. Iterate through that array and **convert** the **JSON** of the user (and address) using the **.map()** method to a **string** like „name: First Last - phone: 1234 - email: example.com - address: In my street 10, 36039 Fulda“.
Ensure you can handle missing attributes or a missing address object from your received response.
Use all those strings and display them in the HTML by attaching the content to a DOM element using
document.getElementById('users').textContent = ...

3) Create an **input field** in your HTML file that allows to type in a **username**. Add a **search button that is on click searching** within all the users (received from the backend). If a user with a matching name was found, show the complete user and address data for all matching users. The result data should be added to an element with id=„result“ in the HTML in the same file / page as the input and search button. Filtering should happen in your frontend only.
Hint: Use the **.filter()** and the **.map()** methods.

Hint: You can access the value of an input field in HTML like this:

```
const userInput: HTMLInputElement =
document.getElementById("username") as HTMLInputElement;

const userName = userInput.value;
```

If Typescript annoys you, you will need this explicit type-casting.

4) Add an **event listener for the input field** in your Typescript file to trigger the search not only on button click but like an „**autocomplete**“ on typing: showing suggestions whenever another **character is typed into** the input field. A reasonable rule of thumb is to not start searching unless at minimum 3 characters are entered.

Again: update the results in another „autocomplete-result“ element in the HTML.

Hint: you can attach an ‚input‘ event listener to your input field using JS/TS. But therefore you must load the javascript after the HTML content that creates DOM elements you want to register event listeners for.

The exact logic of the autocomplete is up to you: you might check if the firstName or lastName of the user contains your typed text, ...